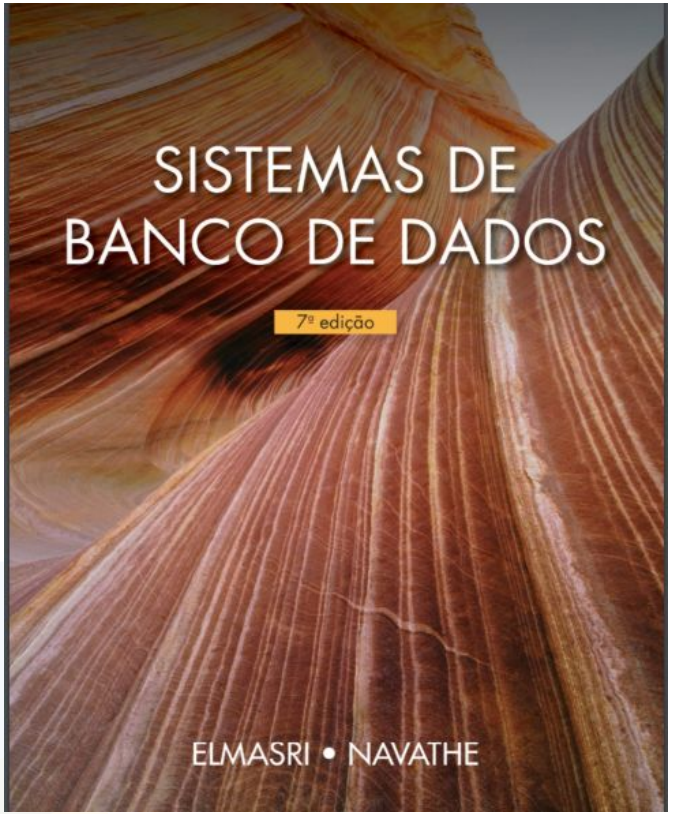
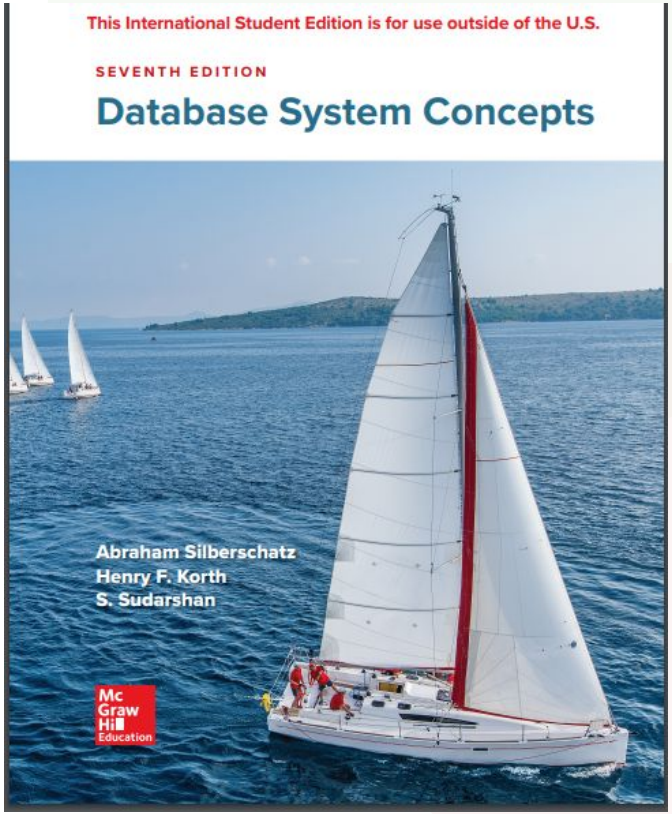
SQL

Consultas avançadas

robespierre.pita@ufba.br

Referências





SQL - Subconsultas ou Consultas Aninhadas

Query 1:

```
SELECT primeiro_nome, ultimo_nome, salario
FROM funcionario
WHERE funcionario.cpf IN
(SELECT cpf_funcionario FROM trabalha_em);
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	João	Silva	30000.00
2	Ronaldo	Lima	38000.00
3	Joice	Leite	25000.00
4	Jorge	Brito	55000.00
5	Fernando	Wong	40000.00
6	Alice	Zelaya	25000.00
7	André	Pereira	25000.00
8	Jennifer	Souza	43000.00

	QUERY PLAN text
1	Hash Join (cost=33.48..37.11 rows=55 width=252)
2	Hash Cond: (trabalha_em.cpf_funcionario = funcionario.cpf)
3	-> HashAggregate (cost=21.00..23.00 rows=200 width=48)
4	Group Key: trabalha_em.cpf_funcionario
5	-> Seq Scan on trabalha_em (cost=0.00..18.80 rows=880 width=4...
6	-> Hash (cost=11.10..11.10 rows=110 width=300)
7	-> Seq Scan on funcionario (cost=0.00..11.10 rows=110 width=3...

WHERE departamento_id = id1 OR departamento_id = id2 OR ...

robespierre.pita@ufba.br

SQL - Subconsultas ou Consultas Aninhadas

Query 2:

```
SELECT primeiro_nome, ultimo_nome, salario
FROM funcionario
LEFT JOIN trabalha_em
    ON funcionario.cpf = trabalha_em.cpf_funcionario;
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	João	Silva	30000.00
2	João	Silva	30000.00
3	Ronaldo	Lima	38000.00
4	Joice	Leite	25000.00
5	Joice	Leite	25000.00
6	Fernando	Wong	40000.00
7	Fernando	Wong	40000.00
8	Fernando	Wong	40000.00
9	Fernando	Wong	40000.00
10	Alice	Zelaya	25000.00
11	Alice	Zelaya	25000.00
12	André	Pereira	25000.00
13	André	Pereira	25000.00
14	Jennifer	Souza	43000.00
15	Jennifer	Souza	43000.00
16	Jorge	Brito	55000.00

	QUERY PLAN text
1	Hash Right Join (cost=12.47..33.63 rows=880 width=252)
2	Hash Cond: (trabalha_em.cpf_funcionario = funcionario.cpf)
3	-> Seq Scan on trabalha_em (cost=0.00..18.80 rows=880 width=48)
4	-> Hash (cost=11.10..11.10 rows=110 width=300)
5	-> Seq Scan on funcionario (cost=0.00..11.10 rows=110 width=3...

robespierre.pita@ufba.br

SQL - Subconsultas ou Consultas Aninhadas

Query 3:

```
SELECT DISTINCT primeiro_nome, ultimo_nome, salario
FROM funcionario
LEFT JOIN trabalha_em
ON funcionario.cpf = trabalha_em.cpf_funcionario;
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	Jorge	Brito	55000.00
2	Ronaldo	Lima	38000.00
3	Joice	Leite	25000.00
4	Alice	Zelaya	25000.00
5	João	Silva	30000.00
6	André	Pereira	25000.00
7	Jennifer	Souza	43000.00
8	Fernando	Wong	40000.00

	QUERY PLAN text
1	HashAggregate (cost=40.23..41.33 rows=110 width=252)
2	Group Key: funcionario.primeiro_nome, funcionario.ultimo_nome, funcionario.sala...
3	-> Hash Right Join (cost=12.47..33.63 rows=880 width=252)
4	Hash Cond: (trabalha_em.cpf_funcionario = funcionario.cpf)
5	-> Seq Scan on trabalha_em (cost=0.00..18.80 rows=880 width=48)
6	-> Hash (cost=11.10..11.10 rows=110 width=300)
7	-> Seq Scan on funcionario (cost=0.00..11.10 rows=110 width=300)

robespierre.pita@ufba.br

SQL - Subconsultas ou Consultas Aninhadas

Query	Vantagens	Desvantagens	Eficiência Geral
Query 1	Simples de escrever e entender. Boa para listas pequenas.	Pode ser ineficiente em tabelas grandes sem índices. Subqueries não otimizadas podem ser custosas.	**Menos eficiente**
Query 2	Geralmente mais eficiente que `IN`. Suporta índices para acelerar o `JOIN`.	Pode trazer duplicatas, aumentando o tamanho do resultado.	**Mais eficiente** (com índices).
Query 3	Remove duplicatas, garantindo resultados mais limpos.	Mais custosa devido à etapa de remoção de duplicatas (`DISTINCT`).	**Intermediária** (dependendo do tamanho do resultado do `JOIN`).

robespierre.pita@ufba.br

SQL - Subconsultas ou Consultas Aninhadas

Forma alternativa de especificar consultas envolvendo relacionamentos entre tabelas

Otimização

- filtragens prévias de dados na subconsulta
  - apenas tuplas/atributos de interesse são combinados com dados da(s) tabela(s) da consulta externa

- Cláusulas de subconsulta
  - nome_atributo [NOT] IN (consulta_SQL)
  - nome_atributo [< | <= | > | >= | < > | !=] ANY (consulta_SQL)
  - nome_atributo [< | <= | > | >= | < > | !=] ALL (consulta_SQL)

robespierre.pita@ufba.br

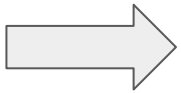
SQL - Subconsultas com IN

Testam a relação de pertinência ou não-pertinência elemento-conjunto

```
select lista_atributos
from tabela1 [...]
where atributo_ou_expressão [NOT] IN (consulta_SQL)
```

Mapeamento para a álgebra relacional

select a_1, ..., a_n
from t_1
where c IN
(select x from t_2
where d > 5)



$$\pi_{a1, ..., an} (t_1 \; \theta \; X \; (\pi_x \; (\sigma_{d > 5} \; (t_2))))$$

$$\theta = t_1.c = t_2.x$$

robespierre.pita@ufba.br

SQL - Exemplos de subconsultas com IN

Álgebra	SQL
π_{nome} ((Médicos θ X θ = Médicos.codm = Consultas.codm (π_{codm} ($\sigma_{data = '06/11/13'}$ (Consultas)))))	Select nome From Médicos Where codm in (select codm from Consultas where data = '06/11/13')
(π_{CPF} (Funcionários)) − (π_{CPF} (Pacientes))	Select CPF From Funcionários Where CPF not in (select CPF from Pacientes)
(π_{CPF} (Médicos)) ∩ (π_{CPF} (Pacientes))	Select CPF From Médicos Where CPF in (select CPF from Pacientes)

robespierre.pita@ufba.br

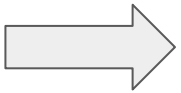
SQL - Subconsultas com ANY

Permitem outras comparações do tipo elemento-conjunto
- testa se um valor é >, <, =, ... que algum valor em um conjunto

```
select lista_atributos
from tabela1 [, ...]
where atributo_ou_expressão [=|<|<=|>|>=|<>| !=] ANY (consulta_SQL)
```

Mapeamento para a álgebra relacional

select a_1, ..., a_n
from t_1
where c **>** **ANY**
(select x from t_2
where d > 5)



$$\pi_{a1, ..., an} (t_1 \; \theta \; X \; (\pi_x \; (\sigma_{d > 5} \; (t_2))))$$

$$\theta = t_1.c > t_2.x$$

robespierre.pita@ufba.br

SQL - Exemplos de subconsultas com ANY

```
SELECT primeiro_nome, ultimo_nome, salario
FROM funcionario
WHERE salario > ANY (
    SELECT salario
    FROM funcionario
    WHERE numero_departamento = 4
);
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	João	Silva	30000.00
2	Fernando	Wong	40000.00
3	Jennifer	Souza	43000.00
4	Ronaldo	Lima	38000.00
5	Jorge	Brito	55000.00

```
SELECT primeiro_nome, ultimo_nome, salario
FROM funcionario
WHERE numero_departamento = 4
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	Alice	Zelaya	25000.00
2	Jennifer	Souza	43000.00
3	André	Pereira	25000.00

WHERE salario > valor1 OR salario > valor2 OR ...

robespierre.pita@ufba.br

SQL - Exemplos de subconsultas com ALL

```
SELECT primeiro_nome, ultimo_nome, salario
FROM funcionario
WHERE salario > ALL (
    SELECT salario
    FROM funcionario
    WHERE numero_departamento = 4
);
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	Jorge	Brito	55000.00

```
SELECT primeiro_nome, ultimo_nome, salario
FROM funcionario
WHERE numero_departamento = 4
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)
1	Alice	Zelaya	25000.00
2	Jennifer	Souza	43000.00
3	André	Pereira	25000.00

WHERE salario > valor1 AND salario > valor2 AND ...

robespierre.pita@ufba.br

Subconsultas com ALL

Operador	Descrição	Use Quando
`IN`	Verifica se o valor está **presente na lista**.	Você precisa comparar por igualdade com valores de uma subquery.
`ANY`	Verifica se a condição é verdadeira para **pelo menos um** valor.	Você precisa verificar uma condição (ex.: maior, menor, igual) para **algum** valor na lista.
`ALL`	Verifica se a condição é verdadeira para **todos os valores**.	Você precisa garantir que a condição seja verdadeira para **todos os valores** da lista.

robespierre.pita@ufba.br

SQL - Comparações Elemento-Elemento

Casos em que a subconsulta retorna apenas um elemento como resultado

- cardinalidade da subconsulta = 1
- não é utilizada nenhuma cláusula de subconsulta neste caso

```
select lista_atributos
from tabela1 [, ...]
where atributo_ou_expressão [=|<|<=|>|>=|<>| !=] (consulta_SQL
com um único elemento)
```

robespierre.pita@ufba.br

SQL - Exemplos de comparações Elemento-Elemento

```
Select nome
From Funcionários
Where salário >
     (Select salário
      From Funcionários
      Where CPF = 22000200002)
```

```
select nome, CPF
from Médicos
where CPF < > 10000100001
and especialidade =
     (select especialidade
      from Médicos
      where CPF = 10000100001)
```

robespierre.pita@ufba.br

SQL - Subconsultas com EXISTS

Quantificador existencial do cálculo relacional

- testa se um predicado é V ou F na subconsulta
- para cada tupla da consulta externa a ser analisada, a subconsulta é executada

```
select lista_atributos
from tabela1 [, ...]
where [NOT] EXISTS (consulta_SQL)
```

```
-- Tem algum depto sem funcionario?
SELECT nome_departamento
FROM departamento d
WHERE EXISTS (
    SELECT 1
    FROM funcionario f
    WHERE f.numero_departamento = d.numero_departamento
);
```

-- A expressão SELECT 1 dentro do EXISTS é apenas simbólica
- o valor não importa, pois o EXISTS só verifica se há linhas retornadas.

robespierre.pita@ufba.br

SQL - Exemplos de subconsultas com EXISTS

Cálculo	SQL
{m.nome | m ∈ Médicos ∧ ∃ c ∈ Consultas (c.data = '06/11/13' ∧ c.codm = m.codm)}	Select nome From Médicos m Where exists (Select * From Consultas Where data = '06/11/13' and codm = m.codm)
{f.nome | f ∈ Funcionários ∧ f.depto = 'pessoal' ∧ ¬ ∃ p ∈ Pacientes (p.CPF = f.CPF)}	Select f.nome From Funcionários f Where f.depto = 'pessoal' and not exists (Select * From Pacientes Where CPF = f.CPF)

robespierre.pita@ufba.br

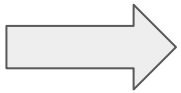
SQL - Subconsulta na Cláusula FROM

Gera uma tabela derivada a partir de uma ou mais tabelas, para uso na consulta externa
otimização: filtra linhas e colunas de uma tabela que são desejadas pela consulta externa

```
select lista_atributos
from (consulta_SQL) as nome_tabela_derivada
```

Mapeamento para a álgebra relacional

select a_1
from (select x
from t_1 where d > 5)
as t_2 join t_3
on $t_3.c = t_2.x$



$$\pi_{a1} (t_3 \theta X \rho_{t2}(\pi_x (\sigma_{d>5} (t_1))))$$

$$\theta = t_3.c = t_2.x$$

robespierre.pita@ufba.br

SQL - Subconsulta na Cláusula FROM

Álgebra	SQL
$\pi_{\text{Médicos.codm, ..., nroa, hora}}$ ((Médicos θ X θ = Médicos.codm = C.codm ρ_C ($\pi_{\text{codm, hora}}$ ($\sigma_{\text{data = '06/11/13'}}$ (Consultas)))))	select Medicos.*, C.hora from Medicos join (select codm, hora from Consultas where data = '06/11/13') as C on Médicos.codm = C.codm
$\pi_{\text{Amb.nroa, andar, capacidade}}$ (ρ_{Amb} ($\pi_{\text{nroa, andar}}$ (Ambulatórios)) θ X θ = Amb.nroa = M_ort.nroa ρ_{MFlo} (π_{nroa} ($\sigma_{\text{cidade = 'Fpolis'}}$ (Médicos))))	select Amb.* from (select nroa, andar from ambulatorios) as Amb join (select nroa from Medicos where cidade = 'Fpolis') as MFlo on Amb.nroa = MFlo.nroa

robespierre.pita@ufba.br

WINDOW

Window

```
FUNCAO() OVER (
    PARTITION BY ...
    ORDER BY ...
    ROWS BETWEEN ...
)
```

```
SELECT
    Numero_departamento,
    AVG(Salario) AS media_salarial
FROM FUNCIONARIO
GROUP BY Numero_departamento;
```

```
SELECT
    Primeiro_Nome, Ultimo_nome,
    Numero_departamento, Salario,
    AVG(Salario) OVER (
        PARTITION BY Numero_departamento
    ) AS media_salarial_departamento
FROM FUNCIONARIO;
```

robespierre.pita@ufba.br

Window

```
FUNCAO() OVER (
    PARTITION BY ...
    ORDER BY ...
    ROWS BETWEEN ...
)
```

OVER() responde 3 perguntas

- “Quem participa?”
  - PARTITION BY
- “Em que ordem?”
  - ORDER BY
- “Quais linhas exatamente?”
  - ROWS BETWEEN

robespierre.pita@ufba.br

Window

	numero_departamento integer	media_salarial numeric
1	4	31000.000000000000
2	1	55000.000000000000
3	5	33250.000000000000

```
SELECT
    Numero_departamento,
    AVG(Salario) AS media_salarial
FROM FUNCIONARIO
GROUP BY Numero_departamento;
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	numero_departamento integer	salario numeric (10,2)	media_salarial_departamento numeric
1	Jorge	Brito	1	55000.00	55000.000000000000
2	Alice	Zelaya	4	25000.00	31000.000000000000
3	Jennifer	Souza	4	43000.00	31000.000000000000
4	André	Pereira	4	25000.00	31000.000000000000
5	Ronaldo	Lima	5	38000.00	33250.000000000000
6	Fernando	Wong	5	40000.00	33250.000000000000
7	João	Silva	5	30000.00	33250.000000000000
8	Joice	Leite	5	25000.00	33250.000000000000

```
SELECT
    Primeiro_Nome, Ultimo_nome,
    Numero_departamento, Salario,
    AVG(Salario) OVER (
        PARTITION BY Numero_departamento
    ) AS media_salarial_departamento
FROM FUNCIONARIO;
```

robespierre.pita@ufba.br

Window - Janela global (sem particionamento)

	primeiro_nome character varying (50)	salario numeric (10,2)	avg numeric
1	João	30000.00	35125.000000000000
2	Fernando	40000.00	35125.000000000000
3	Alice	25000.00	35125.000000000000
4	Jennifer	43000.00	35125.000000000000
5	Ronaldo	38000.00	35125.000000000000
6	Joice	25000.00	35125.000000000000
7	André	25000.00	35125.000000000000
8	Jorge	55000.00	35125.000000000000

```
SELECT
    Primeiro_Nome,
    Salario,
    AVG(Salario) OVER ()
FROM FUNCIONARIO;
```

robespierre.pita@ufba.br

Window - meu primeiro particionamento

	primeiro_nome character varying (50)	salario numeric (10,2)	avg numeric
1	João	30000.00	35125.000000000000
2	Fernando	40000.00	35125.000000000000
3	Alice	25000.00	35125.000000000000
4	Jennifer	43000.00	35125.000000000000
5	Ronaldo	38000.00	35125.000000000000
6	Joice	25000.00	35125.000000000000
7	André	25000.00	35125.000000000000
8	Jorge	55000.00	35125.000000000000

```
SELECT
   Primeiro_Nome, salario, AVG(salario)
OVER (
      PARTITION BY departamento
   )
FROM FUNCIONARIO;
```

robespierre.pita@ufba.br

Window - ranking

	primeiro_nome character varying (50)	salario numeric (10,2)	avg numeric
1	João	30000.00	35125.000000000000
2	Fernando	40000.00	35125.000000000000
3	Alice	25000.00	35125.000000000000
4	Jennifer	43000.00	35125.000000000000
5	Ronaldo	38000.00	35125.000000000000
6	Joice	25000.00	35125.000000000000
7	André	25000.00	35125.000000000000
8	Jorge	55000.00	35125.000000000000

```
SELECT
    Primeiro_Nome,
    salario,
    RANK() OVER (
        PARTITION BY departamento
        ORDER BY salario DESC
    )
FROM FUNCIONARIO;
```

robespierre.pita@ufba.br

Window - soma acumulada

	primeiro_nome character varying (50)	salario numeric (10,2)	sum numeric
1	André	25000.00	75000.00
2	Alice	25000.00	75000.00
3	Joice	25000.00	75000.00
4	João	30000.00	105000.00
5	Ronaldo	38000.00	143000.00
6	Fernando	40000.00	183000.00
7	Jennifer	43000.00	226000.00
8	Jorge	55000.00	281000.00

Uso implicito do frame:

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Todas as linhas com o mesmo valor de ORDER BY pertencem ao mesmo grupo lógico.

```
SELECT
    Primeiro_Nome,
    salario,
    SUM(salario) OVER (
        ORDER BY salario
    )
FROM FUNCIONARIO;
```

Window - soma acumulada

	primeiro_nome character varying (50)	salario numeric (10,2)	sum numeric
1	André	25000.00	75000.00
2	Alice	25000.00	75000.00
3	Joice	25000.00	75000.00
4	João	30000.00	105000.00
5	Ronaldo	38000.00	143000.00
6	Fernando	40000.00	183000.00
7	Jennifer	43000.00	226000.00
8	Jorge	55000.00	281000.00

Uso explícito do frame:

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Todas as linhas com o mesmo valor de ORDER BY pertencem ao mesmo grupo lógico.

```
SELECT
    Primeiro_Nome,
    salario,
    SUM(salario) OVER (
        ORDER BY salario
        ROWS BETWEEN 1 PRECEDING
        AND CURRENT ROW
    )
FROM FUNCIONARIO;
```

Window - exemplo didático “...

robespierre.pita@ufba.br

```
SELECT
    Primeiro_Nome, Ultimo_nome, Salario,
Numero_departamento,
AVG(Salario) OVER (
        PARTITION BY Numero_departamento
    ) AS media_depto,

    RANK() OVER (
        PARTITION BY Numero_departamento
        ORDER BY Salario DESC
    ) AS ranking_salario_depto,

    SUM(Salario) OVER (
        PARTITION BY Numero_departamento
        ORDER BY Salario
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS soma_acumulada_depto

FROM FUNCIONARIO;
```

robespierre.pita@ufba.br

Window - exemplo didático “completo”

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	salario numeric (10,2)	numero_departamento integer	media_depto numeric	ranking_salario_depto bigint	soma_acumulada_depto numeric
1	Jorge	Brito	55000.00	1	55000.000000000000	1	55000.00
2	André	Pereira	25000.00	4	31000.000000000000	2	25000.00
3	Alice	Zelaya	25000.00	4	31000.000000000000	2	50000.00
4	Jennifer	Souza	43000.00	4	31000.000000000000	1	93000.00
5	Joice	Leite	25000.00	5	33250.000000000000	4	25000.00
6	João	Silva	30000.00	5	33250.000000000000	3	55000.00
7	Ronaldo	Lima	38000.00	5	33250.000000000000	2	93000.00
8	Fernando	Wong	40000.00	5	33250.000000000000	1	133000.00

Window com join

Total de horas trabalhadas por funcionário

```
SELECT
    f.Primeiro_Nome,
    f.Ultimo_nome,
    t.Numero_projeto,
    t.Horas,
    SUM(t.Horas) OVER (
        PARTITION BY t.Cpf_funcionario
    ) AS total_horas_funcionario
FROM TRABALHA_EM t
JOIN FUNCIONARIO f
    ON t.Cpf_funcionario = f.Cpf;
```

	primeiro_nome character varying (50)	ultimo_nome character varying (50)	numero_projeto integer	horas numeric (4,1)	total_horas_funcionario numeric
1	João	Silva	1	32.5	40.0
2	João	Silva	2	7.5	40.0
3	Fernando	Wong	20	10.0	40.0
4	Fernando	Wong	2	10.0	40.0
5	Fernando	Wong	3	10.0	40.0
6	Fernando	Wong	10	10.0	40.0
7	Joice	Leite	1	20.0	40.0
8	Joice	Leite	2	20.0	40.0
9	Ronaldo	Lima	3	40.0	40.0
10	Jorge	Brito	20	[null]	[null]

robespierre.pita@ufba.br

Window com subquery

Total de horas trabalhadas por funcionário

```
SELECT Nome_projeto, Primeiro_Nome, Ultimo_nome, Horas
FROM (
    SELECT  p.Nome_projeto, f.Primeiro_Nome, f.Ultimo_nome, t.Horas,
        ROW_NUMBER() OVER (
            PARTITION BY t.Numero_projeto
            ORDER BY t.Horas DESC
        ) AS posicao
    FROM TRABALHA_EM t
    JOIN FUNCIONARIO f
        ON t.Cpf_funcionario = f.Cpf
    JOIN PROJETO p
        ON t.Numero_projeto = p.Numero_projeto
    WHERE t.Horas IS NOT NULL
) AS ranking
WHERE posicao = 1;
```

robespierre.pita@ufba.br

Window com subquery

Funcionário com maior dedicação em cada projeto

	nome_projeto character varying (50)	primeiro_nome character varying (50)	ultimo_nome character varying (50)	horas numeric (4,1)
1	ProdutoX	João	Silva	32.5
2	ProdutoY	Joice	Leite	20.0
3	ProdutoZ	Ronaldo	Lima	40.0
4	Informatização	André	Pereira	35.0
5	Reorganização	Jennifer	Souza	15.0
6	Novosbenefícios	Alice	Zelaya	30.0

Obrigado!!

robespierre.pita@ufba.br